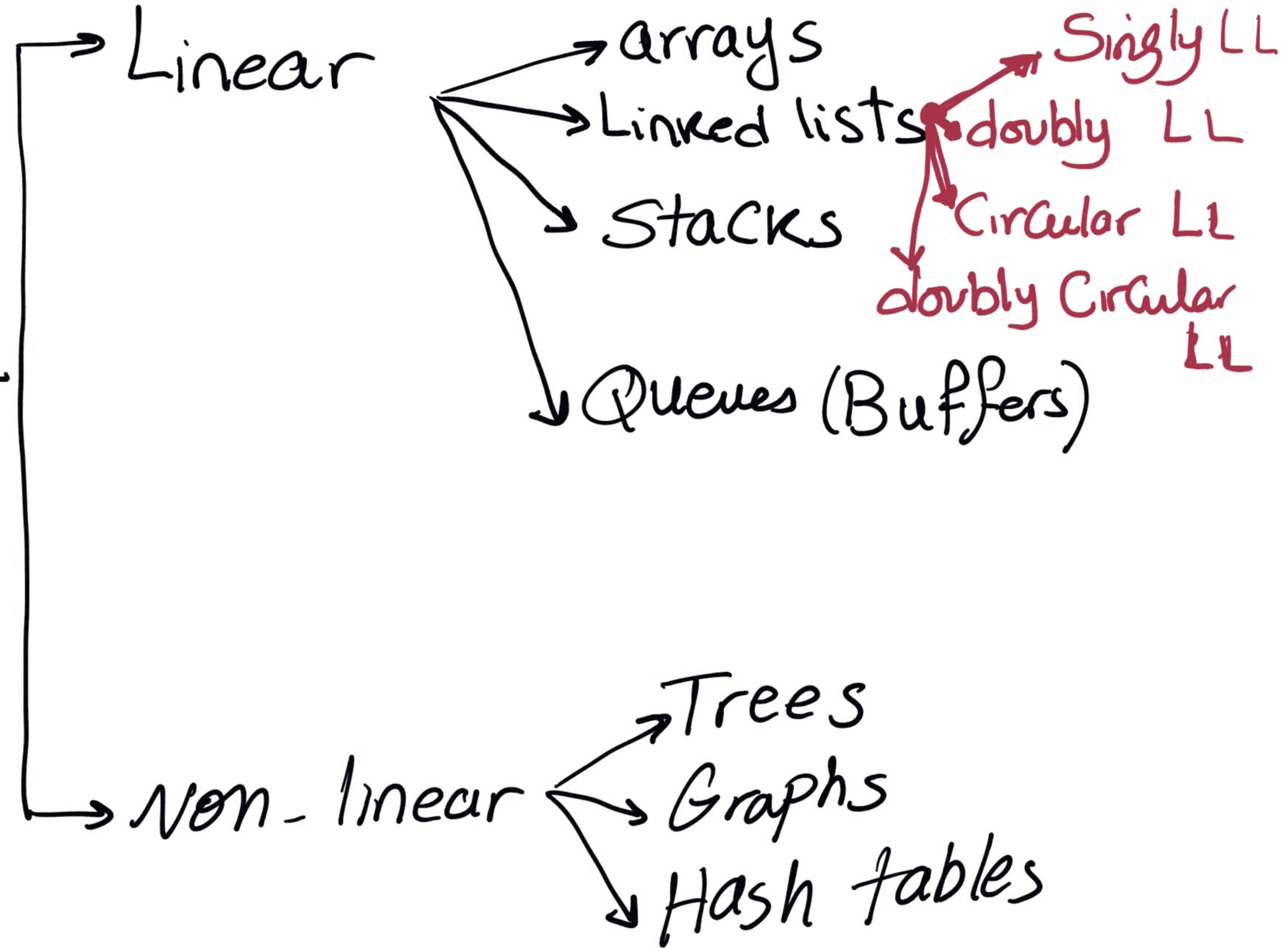


Lecture 2

Linked List

Basic Data Structures



Array is a fixed-size list
of values stored in consecutive places
in memory

Int Scores [3]

1000



4 bytes

12 bytes

Advantages

* Direct access

get the 3rd element

Scores [2]

$O(1)$

Two Major Drawbacks

1- Fixed Size

We cannot insert more than 3 elements

2- lots of wastage of memory

If we declare an array of size 50
but we use only 10 of them

Conclusion

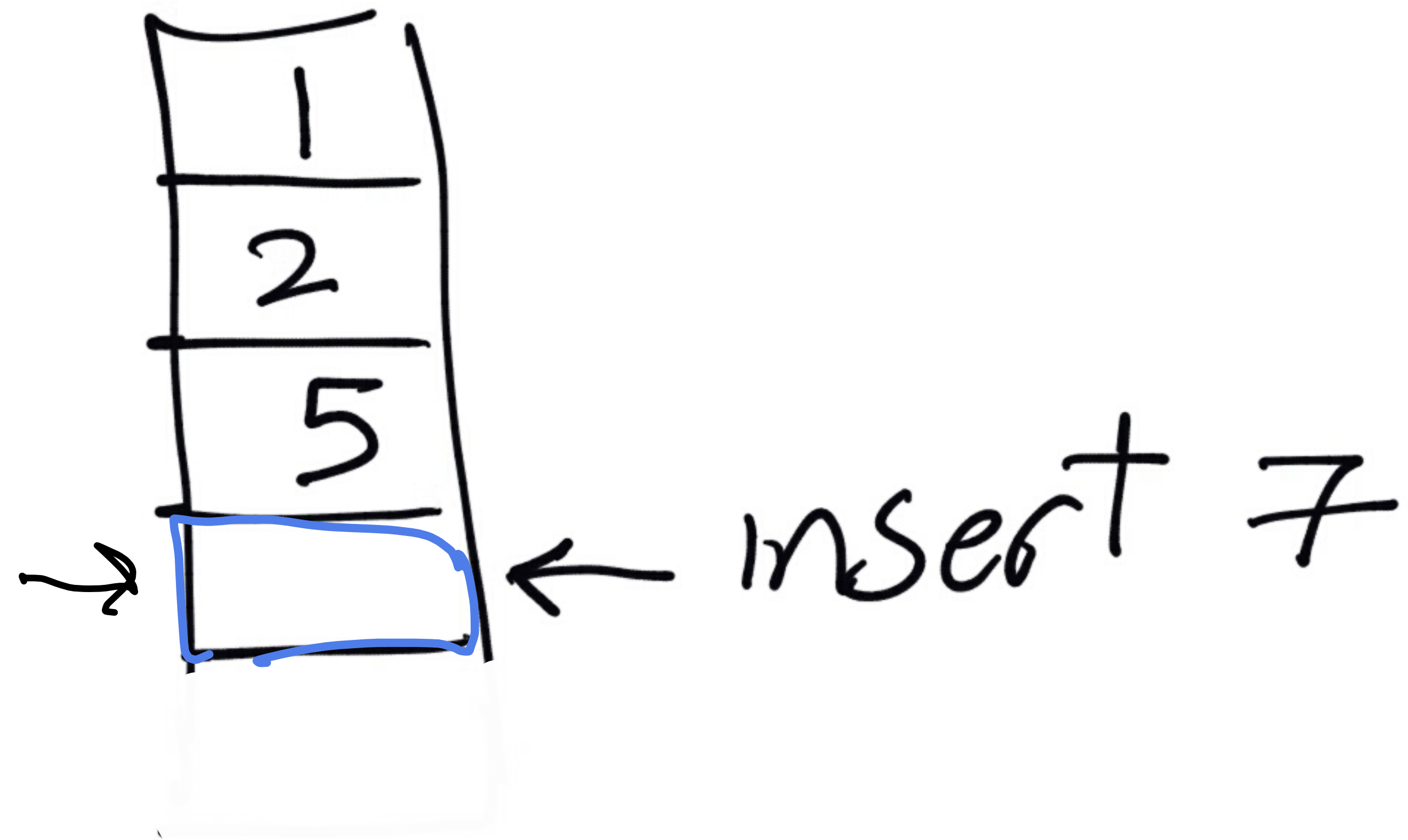
Arrays are fixed size list,

with direct access

Quick in access

Quick when insert/delete at
end

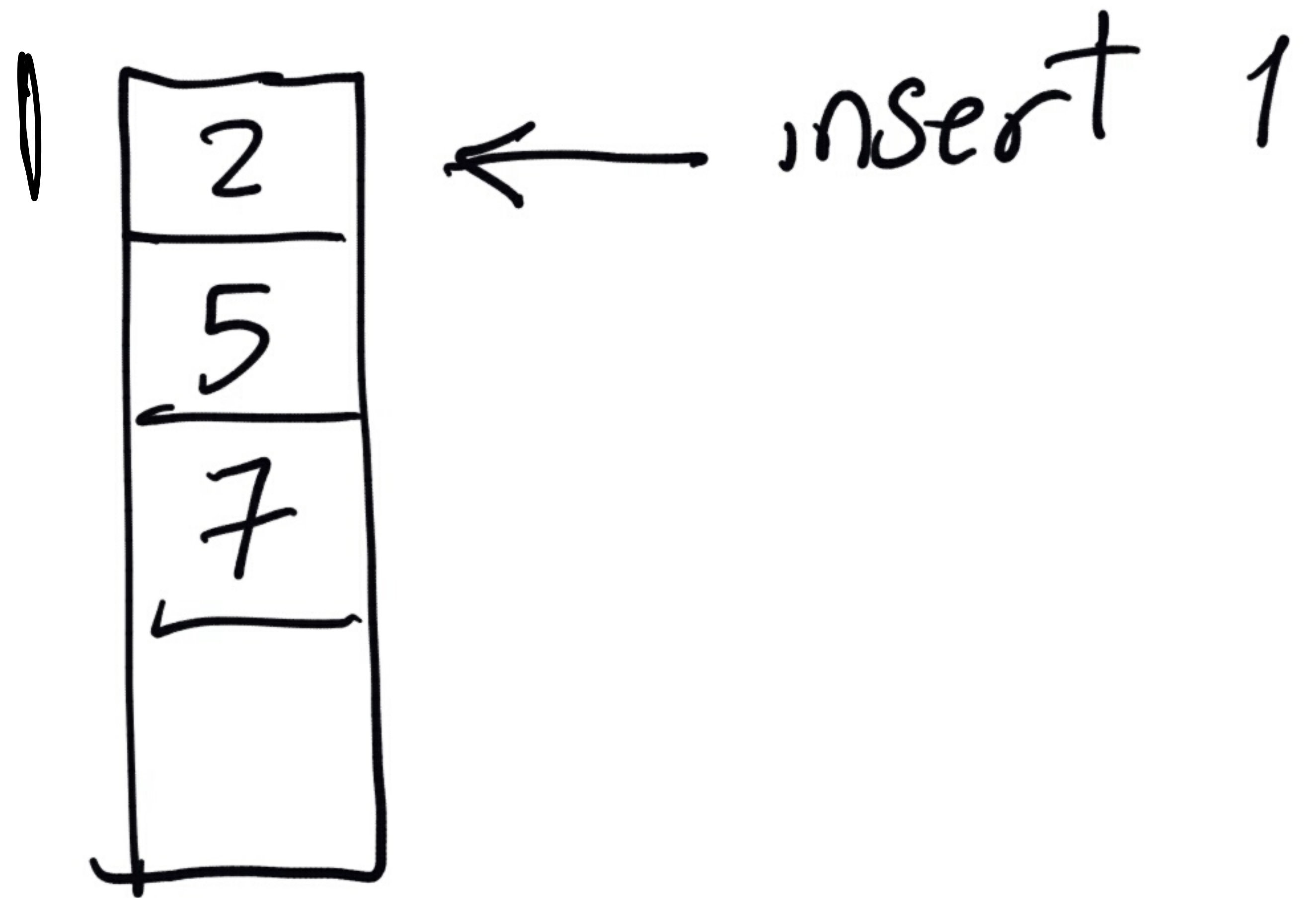
To insert / delete at end



$O(1)$

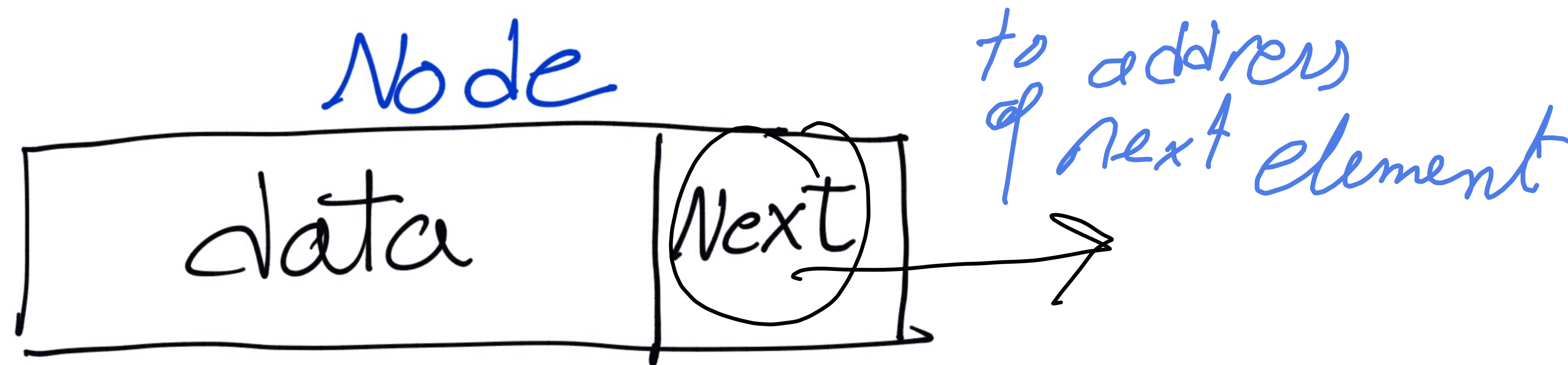
$A[n-1] = \checkmark$

To insert/delete at middle/top



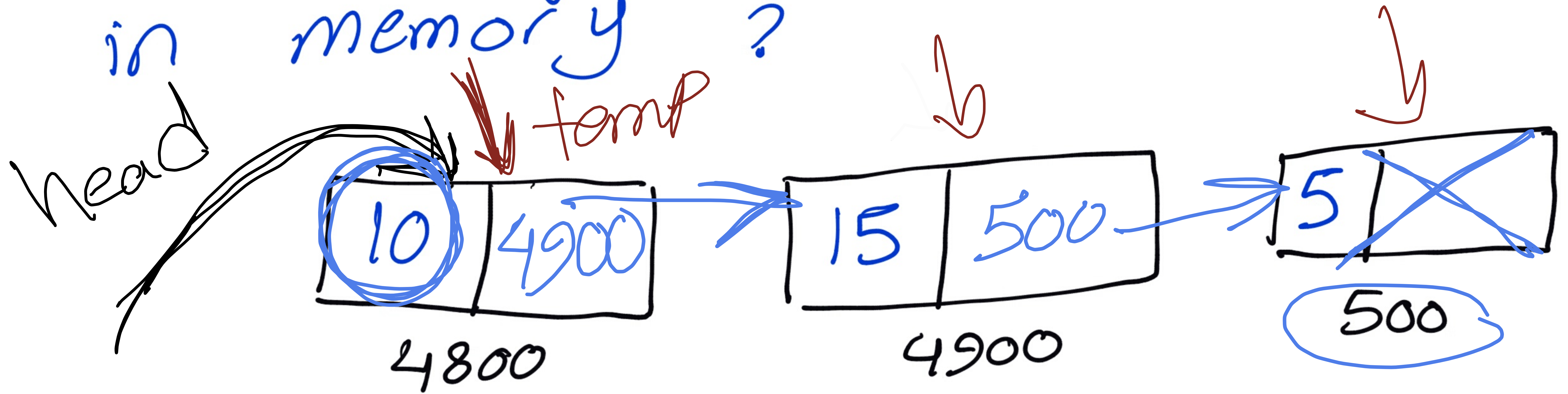
$O(n)$
shift

Linked list is a collection of elements where each element links to next element, not stored in consecutive locations



elements are called **nodes**
each node consists of two
parts data , next

How LL of 10, 15, 5 is stored in memory ?



4800

→ why head ?
→ last node ?

Advantages of LL ?

Memory efficient

- dynamic size

- occupy what you need only

Disadvantages of LL

Memory overhead

Node in LL occupies more memory than array

Sequential Access

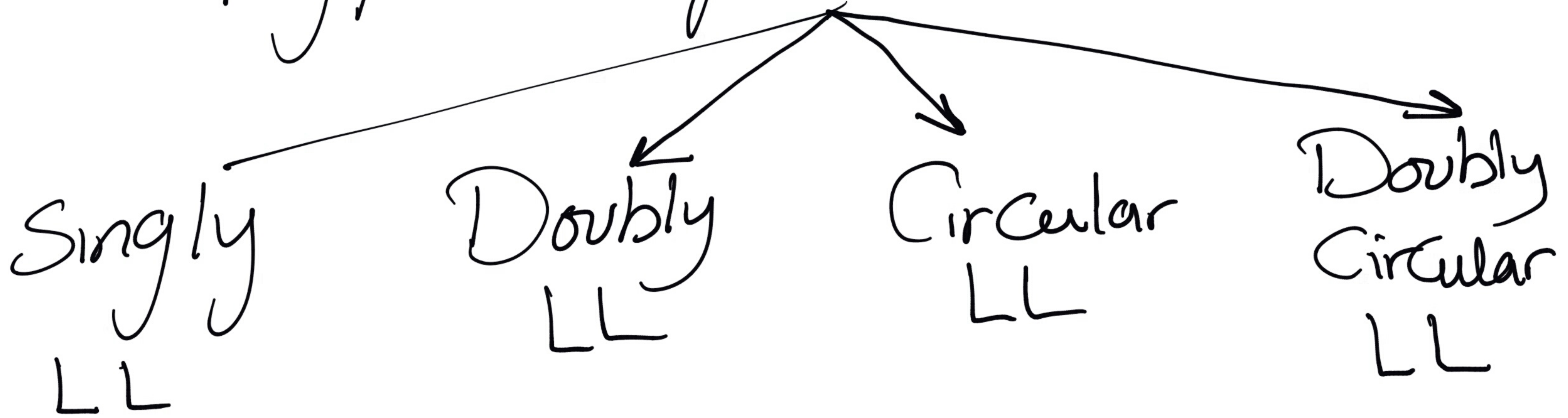
to get n^{th} element always start from head

$O(n)$

Reverse Traversing

backtracking is difficult
than array but In doubly
LL it is easier but
requires more memory

Types of LL

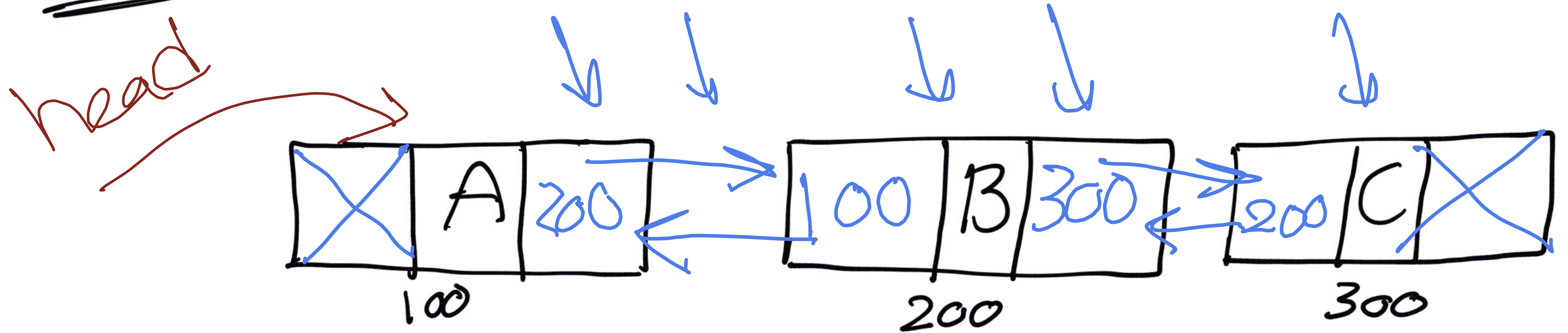


Commonly
Used
LL

Doubly LL



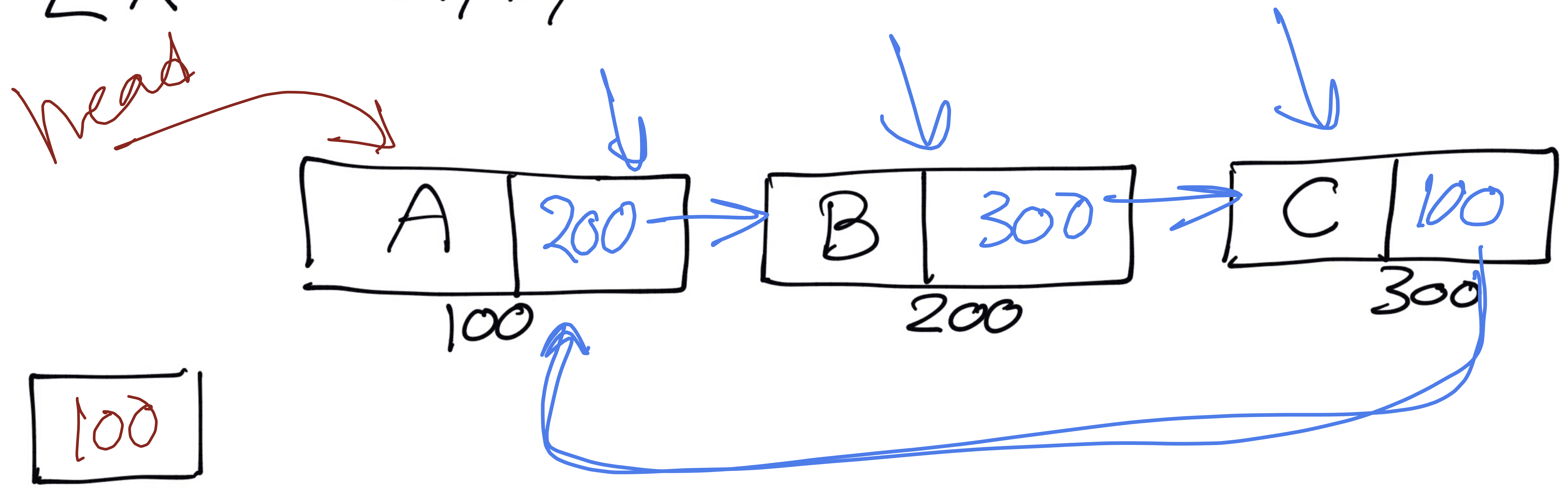
Ex A, B, C



100

Circular LL

EX A, B, C



Basic Operations on Singly LL

- [1] Creating a Node
- [2] Traversing the list
- [3] Searching for element in LL
- [4] Insertion
- [5] Deletion

I Creating a Node

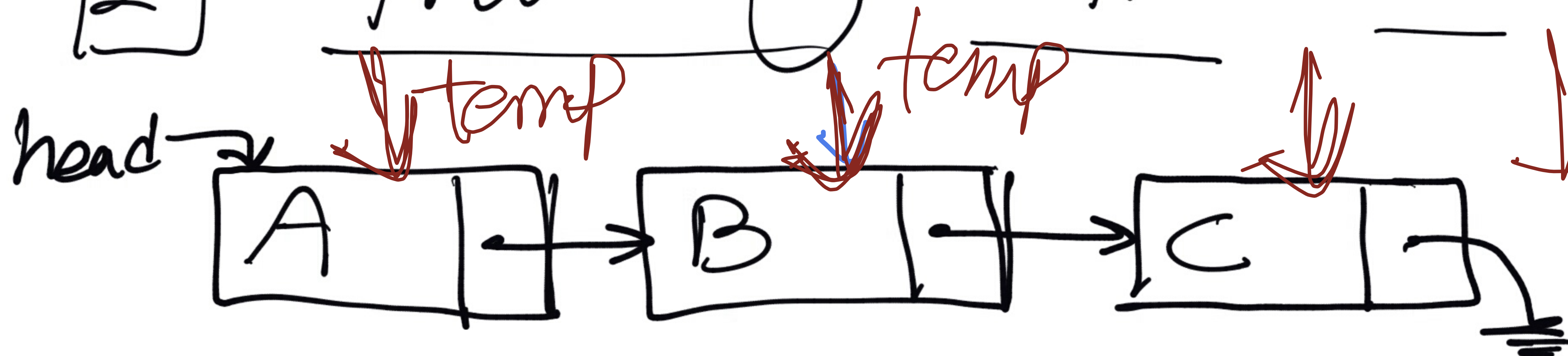
- * Create a node
- * Make a head points to it

$O(1)$

2

Traversing the list

visit each node of LL once for a given purpose



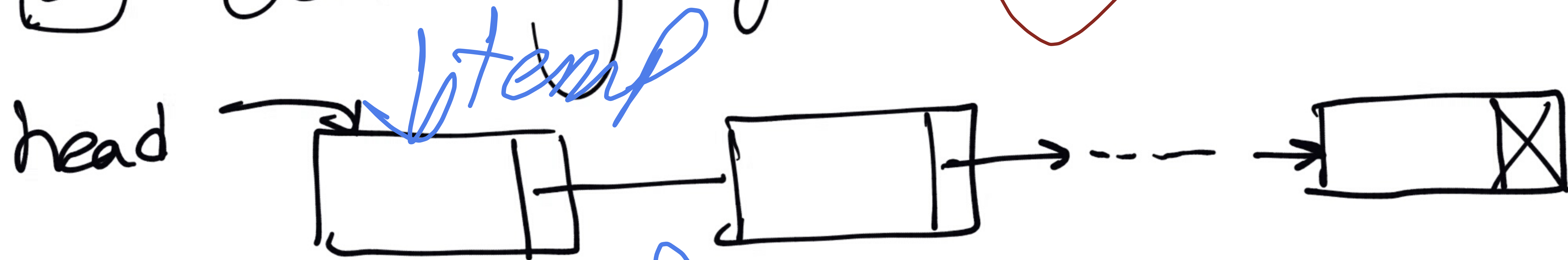
```
temp = head
while (temp != NULL)
{
    print temp.data;
    temp = temp->next;
}
```

Move on

Pseudo Code for traversing LL

$O(1)$

[3] Searching for X in LL



temp = head, found = 0

While (temp != Null)

{ if (temp.data == X) { flag = 1
break; }

temp = temp -> next

}
if (flag == 1) Print "found"
Print "Not found"

Pseudo Code
for
searching

$O(n)$

Best
Case
 $O(1)$

worst
Case
 $O(n)$

avg
case
 $O(n)$

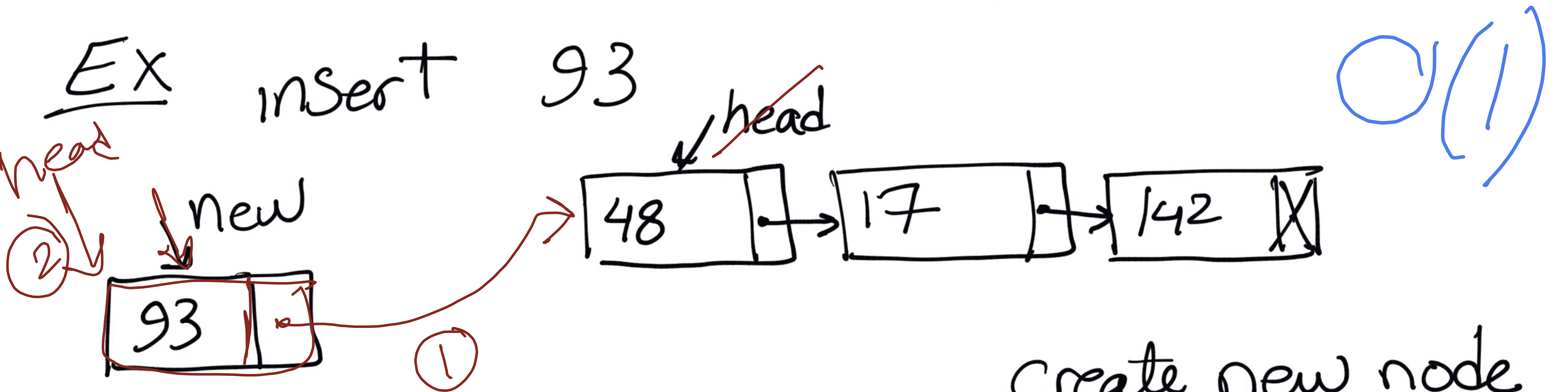
[4] Insertion

i) Insert into empty list

equivalent to Creating
new
Node

$O(1)$

(i) Insert at the top of the list

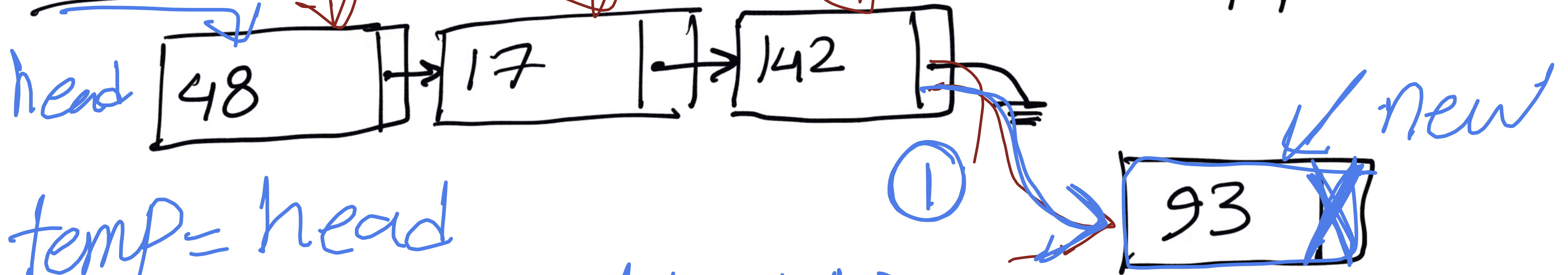


new = Create node
set new.data
new → next = head
head = new

- Create new node
- set node data value
- reconnect links

(iii) Insert at the end of the list
(Append)

Ex insert 93



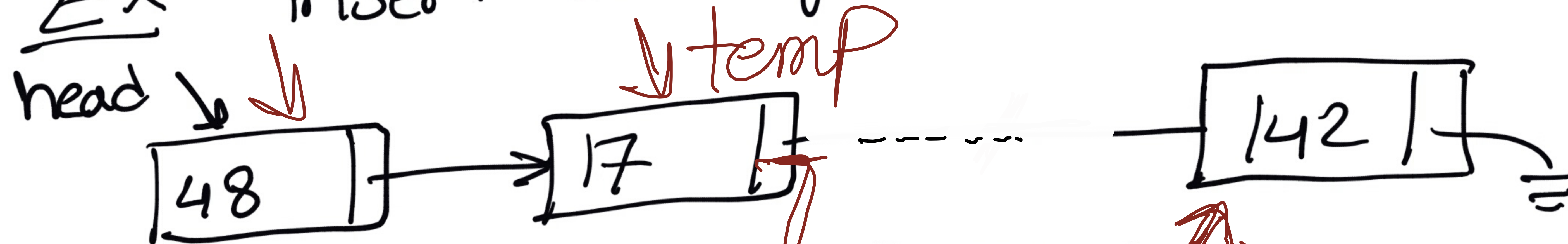
temp = head
while (temp → next != Null)
{ temp = temp → next

$O(n)$

} // temp stops at 142
temp → next = new ; new → next = Null.

(iv) Insert at the middle

Ex insert 93 after 17

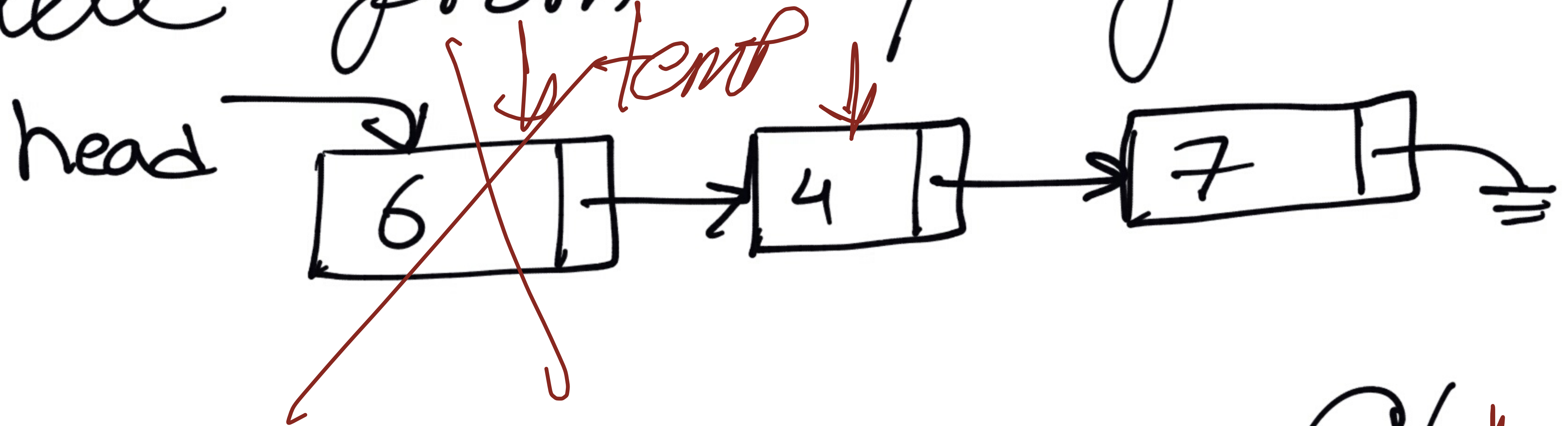


temp = head
while (temp.data != specified value)
{ temp = temp->next;
// temp stops at specified value
new->next = temp->next
temp->next = new

$O(n)$

5] Deletion

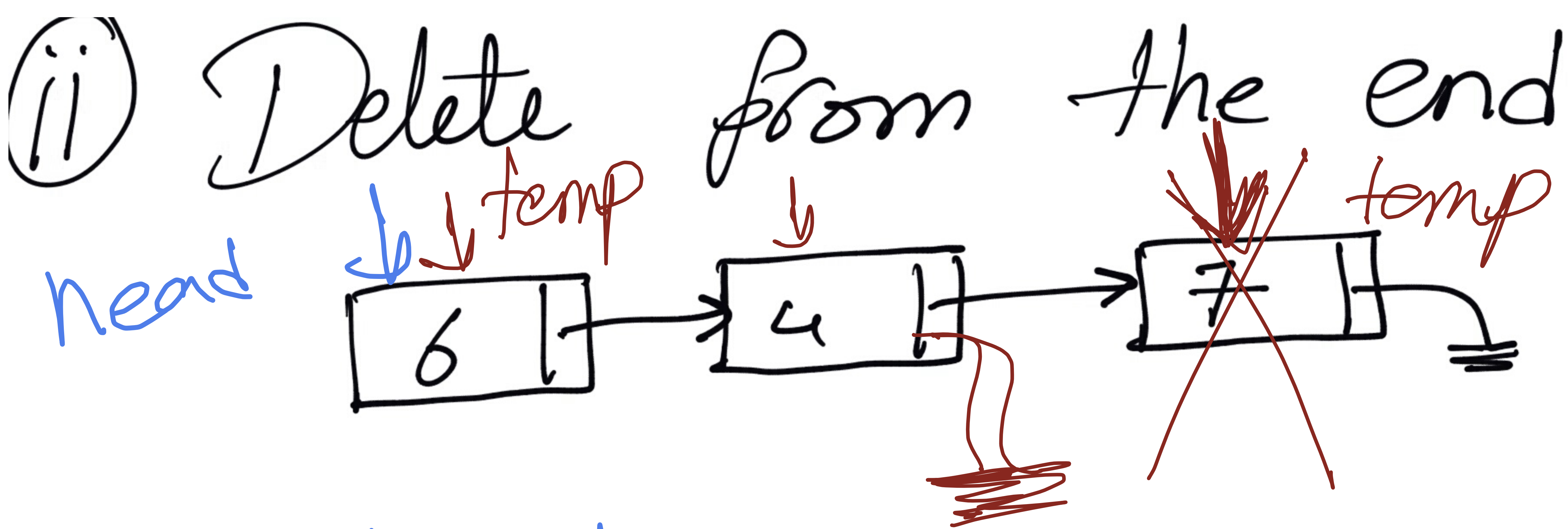
i) Delete from top of list



temp = head
head = temp → next

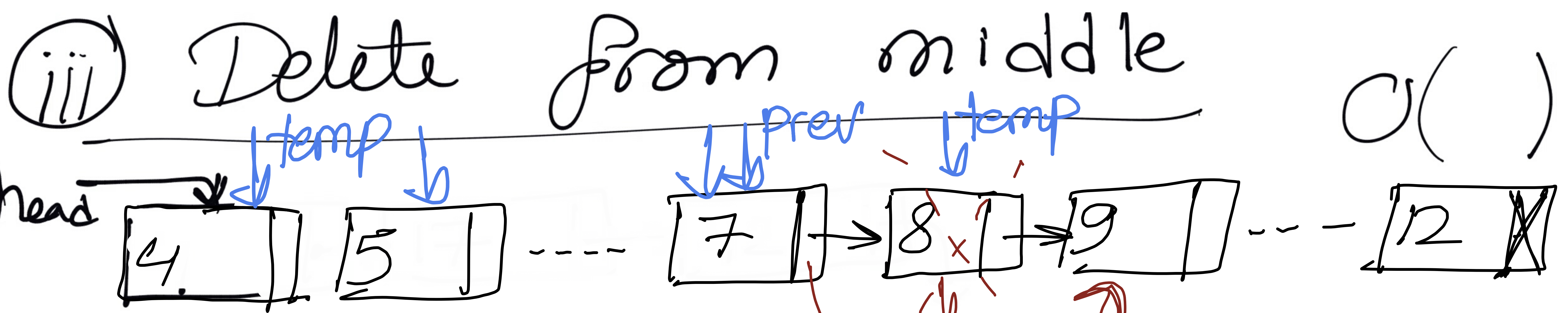
free(temp)

$O(1)$



$O()$

temp = head
while (temp → next ≠ Null)
{ prev = temp
temp = temp → next
} // temp stops at 7, prev stops at 4
prev → next = Null ; free(temp);



```
temp = head
while (temp->data != target value 8)
{
    prev = temp;
    temp = temp->next;
}
// temp stops at 8, prev stops at 7
prev->next = temp->next;
free(temp);
```

Conclusion

	Arrays	LL
Access	$O(1)$	$O(n)$
Search	$O(n)$	$O(n)$

	Arrays	LL
Ins/delete at top	$O(n)$	$O(1)$
Ins/delete at end	$O(1)$	$O(n)$

H.W

you should implement LL
operations using Java

Thank you